



Melbourne Housing Price Prediction

Machine Learning Regression — Workflow & Model Comparison

Built a GUI using Streamlit

Property Features

Number of Rooms

3

-

+

Property Type

House

▼

Distance to CBD (km)

10.00

-

+

Land Size (sqm)

500.00

-

+

Building Area (sqm)

100.00

-

+

Year Built

2000

-

+

Region Name

Eastern Metropolitan

▼

Melbourne Housing Price Predictor

Enter the property details to predict the price using XGBoost model.

Predict Price

Predicted Price: \$1,357,503.62

Project Overview

Property Features

Number of Rooms

3 - +

Property Type

House ▾

Distance to CBD (km)

10.00 - +

Land Size (sqm)

500.00 - +

Building Area (sqm)

100.00 - +

Year Built

2000 - +

Region Name

Eastern Metropolitan ▾

Melbourne Housing Price Predictor

Enter the property details to predict the price using XGBoost model.

Predict Price

Predicted Price: \$1,357,503.62

Goal

Predict Melbourne housing prices using regression models on the `melb_data.csv` dataset.

Workflow

01 Preprocessing

02 Visualization

03 Model Training

04 Evaluation

Libraries & Tools

Data Handling

pandas, numpy

Visualization

matplotlib, seaborn

Preprocessing

*LabelEncoder, StandardScaler,
train_test_split*

Models

XGBoost, Random Forest, Bagging, Decision Tree

Metrics

MSE, RMSE, MAE, R^2

Dataset Exploration

Initial Inspection

Loaded dataset with pandas. Used `df.head()` and `df.info()` to inspect rows, data types, and missing values.

Key attributes: **Rooms**, **BuildingArea**, **Price**, **Suburb**

Missing Value Handling

Car

Filled using mode

BuildingArea

Median by Rooms group

YearBuilt

Median by Suburb

CouncilArea

Most common value by Suburb



BuildingArea constrained to not exceed Landsize.

Data Visualization



Correlation Heatmap

Numerical feature relationships



Histogram

Price distribution



Pie Chart

Property type breakdown



Boxplot

Price vs Rooms



Scatter Plot

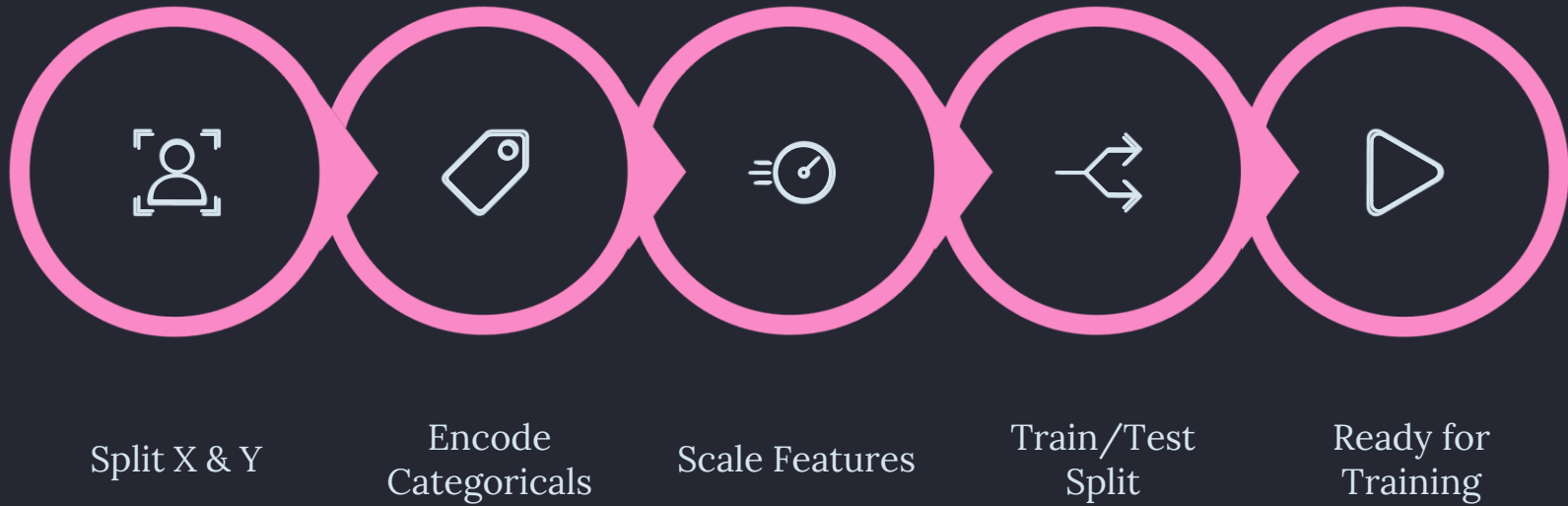
Price vs BuildingArea



Bar Chart

Top suburbs by count

Preprocessing Pipeline



Note: StandardScaler was applied but scaling was ultimately not used during model training.

XGBoost Regressor

XGBoost Regressor

```
model_xgb = XGBRegressor(  
    n_estimators = 800 ,  
    learning_rate=.05,  
    max_depth=4,  
    random_state=42,  
    subsample=0.7,  
    colsample_bytree=0.7,  
    reg_alpha = 1.0,  
    reg_lambda= 3.0,  
    early_stopping_rounds=100  
)  
  
model_xgb.fit(  
    X_train,  
    Y_train,  
    eval_set=[(X_test, Y_test)],  
    verbose=False  
)  
  
Y_pred_xgb = model_xgb.predict(X_test)  
Y_pred_train_xgb = model_xgb.predict(X_train)  
xgb_train_rmse = root_mean_squared_error(Y_train, Y_pred_train_xgb)  
print(f"XGBoost Train RMSE: {xgb_train_rmse}")  
  
xgb_train_r2 = r2_score(Y_train, Y_pred_train_xgb)  
print(f"XGBoost Train R²: {xgb_train_r2}")
```

XGBoost Train RMSE: 191661.731940064

XGBoost Train R²: 0.9107426676196871

Configuration

- *800 estimators, learning rate = 0.05*
- *Regularization + early stopping*

Evaluation

Assessed using RMSE, MAE, MSE, and R^2 .

Expected to deliver the strongest predictive performance among all models.

Bagging & Random Forest

Bagging Regressor

Uses Decision Trees as base estimators. Reduces variance through bootstrap aggregation.

Random Forest

Multiple trees with `max_depth=9`. Adds feature randomness for further variance reduction.

 *Both ensemble methods reduce overfitting compared to a single Decision Tree. Performance measured on both train and test datasets.*

Decision Tree Regressor

Configuration

- `max_depth=10`
- `min_samples_split` and `min_samples_leaf` for regularization

Role

Simpler, interpretable baseline model compared against ensemble methods.

Decision Tree



simple, interpretable,
max_depth=10

Bagging



ensemble,
bootstrap aggregation

Random Forest



ensemble, feature randomness,
max_depth=9

XGBoost



boosting, 800 estimators,
learning_rate=0.05

Results & Conclusion

Ensemble Wins

Ensemble methods generally outperform single Decision Trees in prediction accuracy.

Visual Insights

Visualizations reveal price patterns and feature relationships across Melbourne suburbs.

End-to-End Pipeline

Demonstrates a complete ML regression workflow on real-world data — from preprocessing to model evaluation.

	Model	Train RMSE	Train R ²	Test MSE	Test R ²	Test RMSE	Test MAE
0	XGBoost	191661.73	0.9107	59410543964.81	0.8504	243742.78	155460.13
1	Bagging Regressor	277996.79	0.8122	85120502341.19	0.7857	277996.79	185634.87
2	Random Forest	234658.24	0.8662	86395841514.73	0.7825	293931.70	182810.66
3	Decision Tree	259483.65	0.8364	110835846029.06	0.7210	332920.18	206199.79